



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 10. Кейсы
(/teach/control/stream/view/id/931478155)

PErr10.4 Генератор видео. Часть 2 (new)

Предыдущий урок

(/pl/teach/control/lesson/view?id=341378470)

Следующий урок

(/pl/teach/control/lesson/view?id=341528043)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Что мы умеем

- ✓ Оборачиваем AI-скрипт генерации видео в удобный пользовательский интерфейс: Telegram-бот и Flask-сайт.
- ✓ Создаём Telegram-бота через Telegram и настраиваем токен через BotFather.
- ✓ Подключаем Proxy API и работаем с моделями OpenAI через единый API-клиент.
- ✓ Реализуем отслеживание статуса генерации: очередь, прогресс-бар, обработка, скачивание, ошибки.
- ✓ Разделяем архитектуру на backend (логика генерации) и frontend (бот и сайт).

- ✓ Настраиваем Flask-приложение с роутами: запуск задачи, проверка статуса, скачивание видео.
- ✓ Генерируем уникальные task_id (UUID) для идентификации задач на сайте.
- ✓ Запускаем проект на сервере через SSH, virtualenv и screen для работы 24/7.
- ✓ Публикуем проект в GitHub и готовим его к демонстрации клиентам и в портфолио.

Результат дня

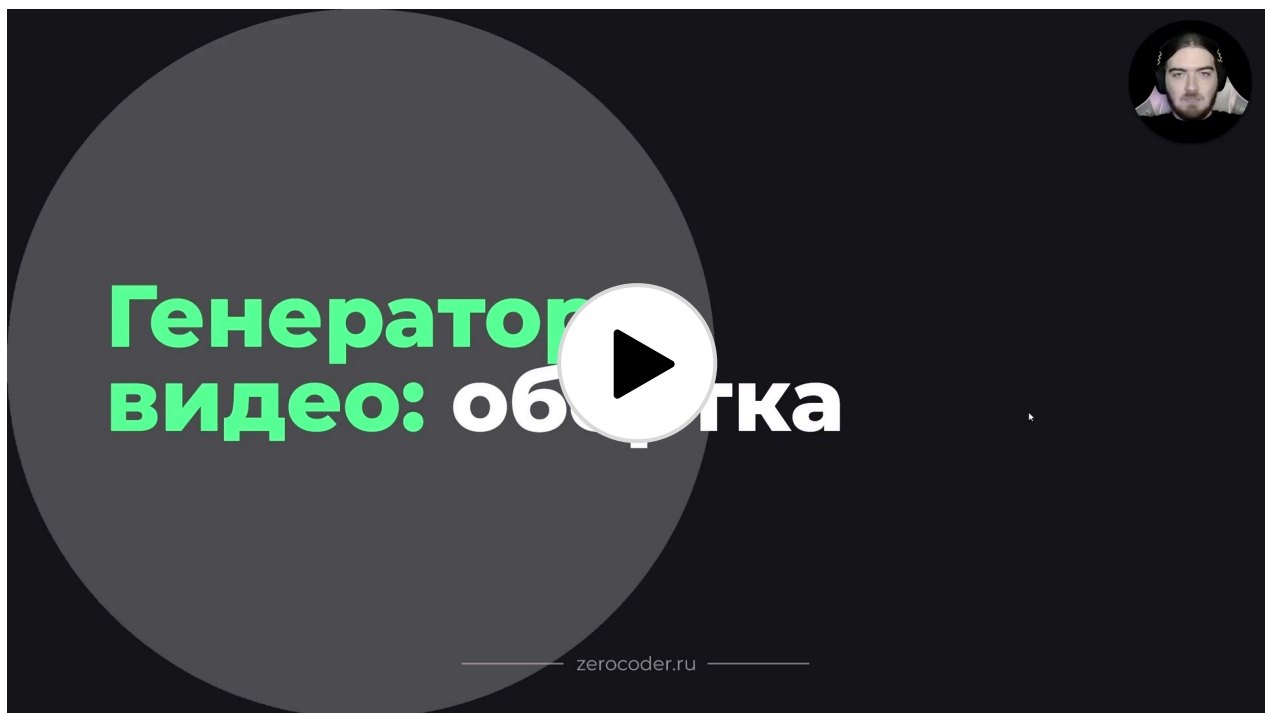
Ты научишься превращать AI-скрипт в полноценный цифровой продукт: создашь Telegram-бота и веб-сайт на Flask с прогресс-баром и системой статусов, развернёшь проект на сервере и запустишь его в режиме 24/7, чтобы любой пользователь мог генерировать видео через удобный интерфейс без работы с терминалом.

▶ Время чтения и просмотра: ~ 2 часа

🕒 Время выполнения задания: ~ 2 часа

Теория на сегодня

1. Введение. Цели урока



▼ Введение. Цели урока

Мы продолжаем изучение курса промпт-инжиниринга и двигаемся дальше в блоке уроков, посвящённых реализации конкретных кейсов — практических примеров, которые можно использовать для наполнения портфолио или просто чтобы набить руку.

В прошлом уроке мы вместе сделали генератор видео. Он работал в формате скрипта и запускался исключительно из командной строки. В целом всё было функционально: мы могли загрузить промпт, получить и скачать готовое видео, посмотреть его и дальше с ним работать.

Но мы также обсуждали, что такой формат совершенно не подходит для конечного пользователя. Для клиента работа через терминал и командную строку выглядит сложно, пугающе и в целом неудобно.

Намного проще, когда есть поле ввода, кнопки, меню навигации — когда вся система управляется не через команды, а через понятный интерфейс: предзаписанные кнопки и поля для ввода данных.

Реальный пользователь не будет скачивать скрипт, открывать терминал и разбираться, что куда вводить. Этим занимаются разработчики. Обычному человеку это просто не нужно.



Поэтому наша задача на сегодняшний урок — взять то, что мы реализовали в прошлом уроке, и обернуть это в удобный интерфейс: в красивый Flask-сайт и в Telegram-бота, с которыми

будет приятно и просто работать клиенту.

Мы разберём, как получать токен для Telegram-бота, пройдемся по всему процессу и в конце запустим и сайт, и бота на сервере, чтобы они работали 24/7 и всегда могли отвечать на запросы пользователей.

Цели сегодняшнего урока:

- Повторим, как делать запросы к разным нейросетям: ещё раз пройдемся по теории Proxy API и похожих сервисов, обсудим, как мы их используем на практике;
- Обернём наше приложение в понятный пользовательский интерфейс: сделаем кнопки, поля, аккуратный внешний вид — всё то, что соответствует стандартному современному продукту;



Когда мы говорим про такие «обёртки» для функционала, обычно выделяют три основные формы:

- Telegram-бот;
- сайт;
- приложение.

Это три базовых способа дать пользователю доступ к возможностям системы без требований к техническим навыкам. Не нужно уметь программировать — достаточно уметь читать и нажимать кнопки. Такой интерфейс интуитивно понятен и давно стал стандартом для любого цифрового продукта.

- Логически завершим второй кейс:
 - обернём код в Telegram-бота и Flask-приложение;
 - выгрузим всё это на GitHub;
 - разберём процесс развёртывания: что делать на сервере, как запускать проект, как пройти путь от репозитория до работающего сервиса.

▼ Краткий обзор пройденного материала

Начнём с повторения основ прошлого урока.

Мы уже научились работать с разными API через единый сервис. В нашем случае это Proxy API, но подобных сервисов существует очень много; можно выбирать любой, который удобен или к которому есть доступ.



Если у тебя есть доступ к западным сервисам, можно использовать аналоги вроде Open API — по сути, они делают то же самое. Таких сервисов много: есть крупные, есть маленькие, но смысл у них одинаковый: это удобный интерфейс, через который мы можем обращаться к любой нужной API-системе (в нашем случае — к AI-моделям).

Мы отправляем запросы, получаем ответы, обрабатываем информацию и используем её дальше. Например, в нашем проекте: отправили запрос, получили видео, выложили его на сайт или отправили в Telegram-бот.

Главное преимущество в том, что мы **снимаем с себя необходимость каждый раз настраивать десятки разных интеграций**. Если в проекте используются модели от разных компаний, нам не нужно подключать каждую API отдельно, а работаем через один централизованный сервис управления.

Через такой API мы:

- улучшим промпты для видео: сделаем дополнительный текстовый запрос для улучшения исходных данных, из которых будет формироваться видео;
- сгенерируем видео;
- можем генерировать фото, текст и т.д.;
- получим доступ ко всем современным моделям, которые могут быть интересны: к новым функциям OpenAI, к моделям Google, Anthropic и др.

Подобные сервисы — это максимально удобный инструмент для повседневной работы. Фактически это прокладка между нами и реальными вычислительными системами, которые дают мощь, алгоритмы и то, что мы называем ИИ и LLM-моделями.

2. Возможности Proxy API



Возможности ProxyAPI

- 1. Доступ ко всем современным моделям**
Одно API - множество функций
- 2. Работа с сервисами, которые не умеют работать напрямую**
Возможность работать с OpenAI, Anthropic, Groq, Llama Labs и тд
- 3. API доступ**
Возможность работать с моделями по удобной API (набор удобных правил, инструментов и протоколов). Единый API для всех наших проектов

zerocoder.ru

▼ Возможности Proxy API

Возможности Proxy API, которые мы используем или можем использовать, можно свести к нескольким ключевым пунктам.

- 1. Доступ ко всем современным моделям.** Это могут быть OpenAI, Google, Perplexity, DeepSeek и многие другие. Таких моделей сейчас очень много, и самое важное — мы всегда получаем доступ к самым новым и современным решениям на рынке.



Практически все хотят использовать самые сильные алгоритмы, которые существуют, в том числе потому, что со временем они становятся дешевле. Если сравнить, например, GPT-5.2 mini и GPT-4, то по цене запросов они будут плюс-минус одинаковыми, но при этом у более новой модели будут явные преимущества: она быстрее, умнее и в целом эффективнее.

Это происходит просто потому, что модель стала современнее: что-то оптимизировали, что-то упростили, где-то ускорили обработку запросов. Технологии не стоят на месте, и в большинстве случаев самые новые модели оказываются и самыми лучшими — они снижают стоимость и повышают производительность (хотя, конечно, всегда стоит смотреть на конкретную модель и конкретный кейс).

2. **Работа с сервисами, которые не хотят работать напрямую.** Среди них OpenAI, Google, 11Labs и многие другие. Таких сервисов сейчас очень много.

Через Proxy API запрос будет обрабатываться на том же самом сервере, что и при прямом обращении. Например, если мы отправляем запрос в OpenAI через Proxy API, фактическая обработка всё равно происходит на стороне OpenAI.

Для нас итог одинаковый, а вот выбор инструмента зависит уже от других факторов: удобства, доступности, ограничений по регионам и так далее.

Хороший пример — сервис 11Labs. Это один из самых мощных генераторов голоса на рынке, особенно в задачах speech-to-text и text-to-speech. Но из России получить к ним доступ напрямую крайне сложно. Обычные способы подключения почти никогда не работают, и нужно сильно постараться, чтобы вообще зайти на их сайт.

Через такие сервисы, как Proxy API или GenAPI, мы можем отправлять запросы даже в те нейросети, которые очень жёстко защищены и блокируют доступ с нашей территории, и это реально большое преимущество.



При этом в Proxu API, например, нет 11Labs, и это нормально. Таких прокси-сервисов очень много. Если нам нужен конкретный AI-сервис, которого нет в Proxu API, почти всегда можно найти другой подобный сервис, где он есть.

Перечислять такие сервисы бессмысленно: новые появляются буквально каждые одну-две недели. Рынок очень динамичный, и через месяц список лидеров может полностью измениться. Это нормальное состояние для этой сферы.

Поэтому мы обычно ориентируемся на те сервисы, которые нам подходят и нравятся. Proxu API здесь просто хороший пример стабильной платформы, которая уже несколько лет работает и даёт доступ к самым популярным моделям — OpenAI, Google, Anthropic и другим. А специфические инструменты вроде 11Labs нужны реже и в узких задачах, например, когда требуется максимально человекоподобный голос.

3. Универсальный API-доступ и клиент-серверная логика. API — это набор правил и инструментов, который позволяет выстраивать модель клиент-сервер.

В нашем случае:

- сначала мы выступаем клиентом по отношению к серверам OpenAI или других сервисов: отправляем запрос, получаем ответ;
- а когда мы разворачиваем всё на собственном сервере, уже наш сервер становится промежуточным клиентом:
 - он получит запрос от пользователя;
 - отправит его дальше в нейросеть;
 - и вернёт результат обратно пользователю.

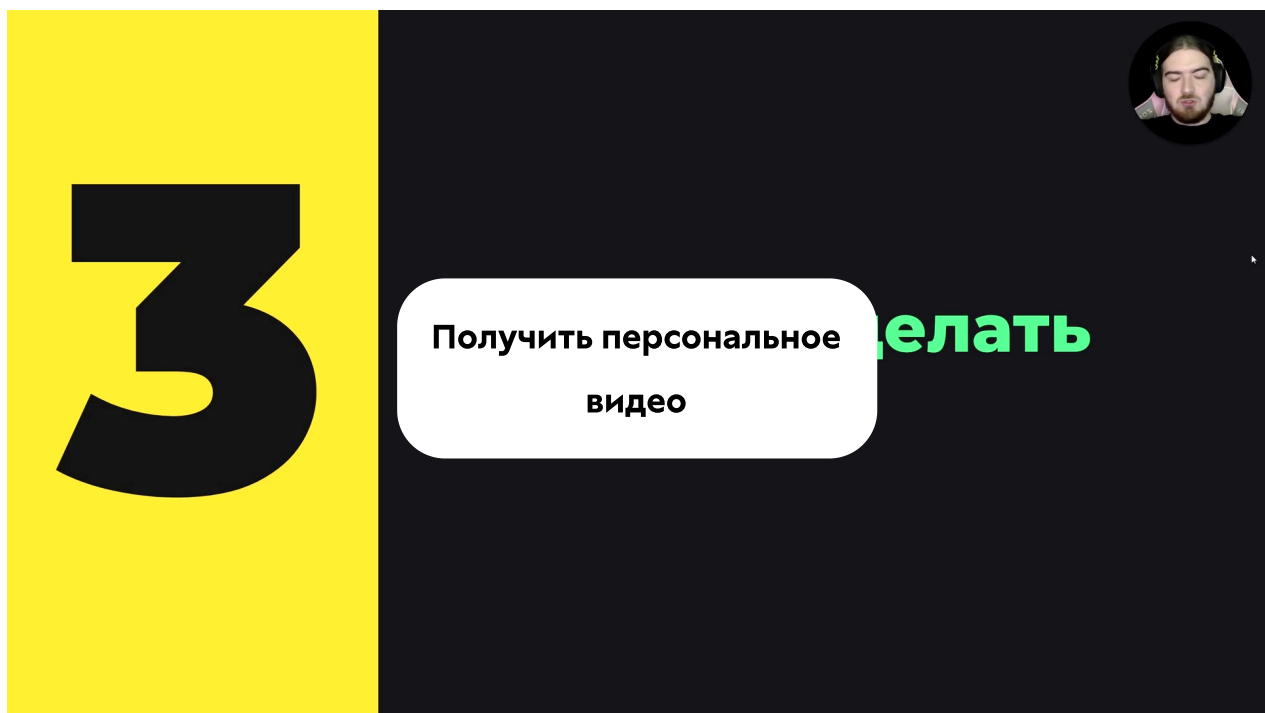
API в целом встречается буквально везде. Это стандарт обмена данными между программами. Его легко сравнить с тем, как мы пользуемся сайтами:

- например, мы хотим посмотреть товары на маркетплейсе: заходим на сайт, вводим запрос, выбираем категорию и видим результат;
- если мы хотим сделать то же самое изнутри кода, мы используем API: отправляем на сервер запрос с параметрами (категория, название, фильтры) и получаем ответ.

Только вместо HTML-страницы мы получаем структурированные данные, которые удобно обрабатывать программно. Это важно, потому что код не умеет «догадываться», если что-то поменялось визуально. Ему нужен стабильный, предсказуемый формат.

API **гарантирует, что ответы всегда приходят в одном и том же виде**. Нам не нужно каждый раз подстраиваться или заново разбирать структуру данных — всё работает по понятным и постоянным правилам.

3. Что будем делать сегодня



▼ **Задача 1: реализация ТГ-бота для работы со скриптом генерации**

Сегодня мы обернём наше приложение, весь существующий функционал, в красивую и удобную оболочку. Новых функций добавлять не будем — наша задача не расширить возможности системы, а сделать взаимодействие с ней проще и приятнее.

Обычному пользователю неудобно работать с терминалом, скриптами и прочими техническими форматами. Он не будет разбираться в командной строке и запускать что-то вручную. Зато с

сайтом или Telegram-ботом он будет работать спокойно и без напряжения: зашёл, нажал кнопки, получил результат.

Первая наша задача на сегодня — это реализация Telegram-бота для работы со скриптом генерации. Мы реализуем Telegram-бота, через которого можно будет:

- делать запросы для создания видео;
- отслеживать статус генерации видео: мы сможем наблюдать процесс в виде прогресс-бара, то есть полосы загрузки, которая будет постепенно заполняться;
- смотреть видео в удобном формате: не просто файл, а полноценное видео, которое можно посмотреть прямо внутри приложения, переслать или скачать.

Всё это мы уже делали в прошлом уроке, только теперь этот функционал будет доступен прямо в Telegram.

▼ Принцип минимализма и удобства интерфейса

Telegram-боты существуют уже давно, все знают, как с ними работать, и для обычных пользователей это максимально понятный интерфейс.

Когда мы будем делать собственные проекты и кейсы для портфолио, это всегда стоит держать в голове. С терминалом работают разработчики и программисты. Для них это мощный и удобный инструмент. Но для обычного человека это тёмный лес, в который он просто не пойдёт.

Это хороший принцип на будущее: **стремиться создавать интерфейс, через который обычный пользователь сможет легко взаимодействовать с системой**, при этом нажимая как можно меньше кнопок. Пользователи не любят нажимать кнопки, часто путаются и не понимают, что именно нужно сделать.

 Минимализм — наш лучший друг. **Чем меньше деталей и элементов интерфейса, тем проще человеку разобраться** и тем комфортнее ему пользоваться системой.

В последнее время все системы стремятся к минимизации действий пользователя. Чем меньше кликов и вводов текста — тем лучше.

Это легко увидеть на примере повседневных сервисов. Например, система быстрых платежей: не нужно вводить номер карты, не нужно вводить телефон — достаточно отсканировать QR-код и нажать одну кнопку.



Конечно, существуют и терминальные, сложные интерфейсы — например, для мониторинга серверов, логов, метрик и других профессиональных задач. Но это инструменты для разработчиков, и ими пользуются те, кто умеет с ними работать. Для обычного клиента мы всегда стремимся к простоте. **Наш идеал — это одна кнопка до покупки.**

▼ **Задача 2: реализация сайта для работы со скриптом генерации**

Вторая задача на сегодня — это реализация сайта для работы со скриптом генерации.

Через сайт мы точно так же будем:

- отправлять запросы на создание видео;
- отслеживать статус генерации, но уже в формате веб-интерфейса;
- скачивать видео.

По сути, у нас есть единый функционал, который мы просто делим на два канала взаимодействия: через сайт и через Telegram-бот.

Это будет простой сайт, внутри которого мы вводим промпт, нажимаем кнопку «Начать генерацию», ждём, пока процесс дойдёт до конца, и после этого скачиваем получившееся видео — для сохранения, дальнейшей работы или дополнительной обработки.

Функционал при этом остаётся тем же самым. Мы ничего не будем менять в логике сервиса: весь кейс построен на генерации видео, и все запросы по-прежнему идут через один и тот же скрипт. Мы посмотрим, как этот скрипт немного переделать, где-то что-то заменить, где-то чуть изменить, а дальше уже будем взаимодействовать с ним через эти обёртки.



Даже для нас как для разработчиков такой формат оказывается удобнее. Нам не нужно каждый раз обращаться к коду и что-то править — мы можем работать как обычный пользователь: зашли на сайт, ввели промпт, нажали кнопку, подождали и получили результат.

После того как мы всё проверим, разберёмся, как это работает, и перенесём проект на сервер, такой скрипт можно использовать как полноценный генератор видео из любой точки: зайти на сайт или в бота и сгенерировать нужный контент.

При желании всю эту систему можно заменить на что угодно: на ИИ-агента, на другой сервис, на более сложную архитектуру — принцип останется тем же.

▼ Backend и frontend в проекте

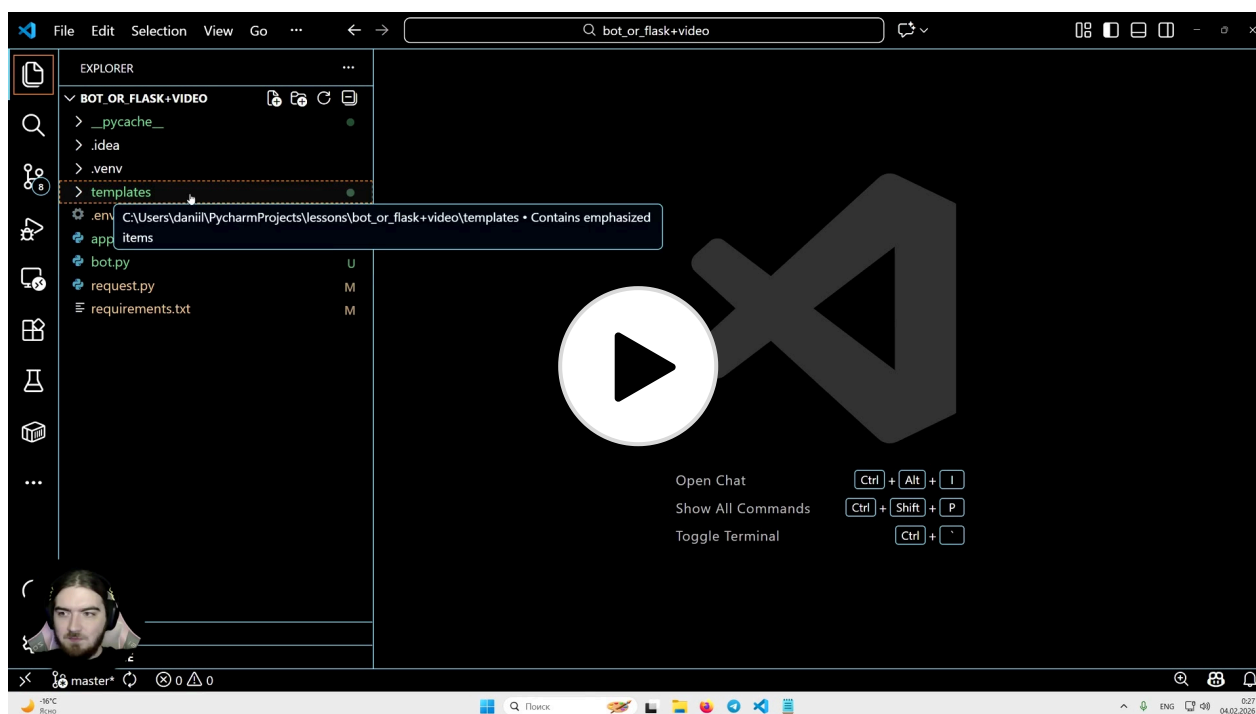
В проекте удобно выделить две основные части разработки: backend и frontend.

- **Backend** — это логика и функционал. В нашем случае backend — это сам скрипт генерации, который отправляет API-запросы через Proxy API в OpenAI или Sora и получает результат.
- **Frontend** — это Telegram-бот и сайт. Это точка соприкосновения с пользователем, удобный интерфейс для ввода данных и получения ответа.

Frontend принимает команды от пользователя и передаёт их на backend, а backend уже выполняет всю работу: отправляет запросы, обрабатывает их, генерирует результат и возвращает его обратно пользователю.

Итак, перейдём к практике: посмотрим код, напомним его, реализуем, запустим, а затем перенесём всё это на сервер, чтобы система работала 24/7 и была доступна из любой точки мира.

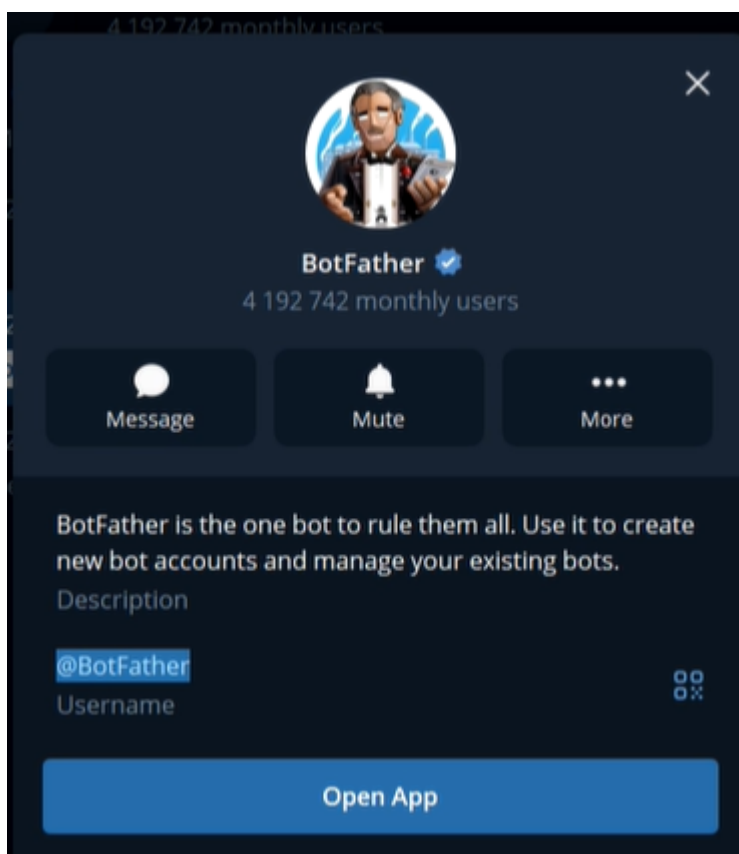
4. Реализация ТГ-бота для работы с генерацией видео



▼ Создание ТГ-бота

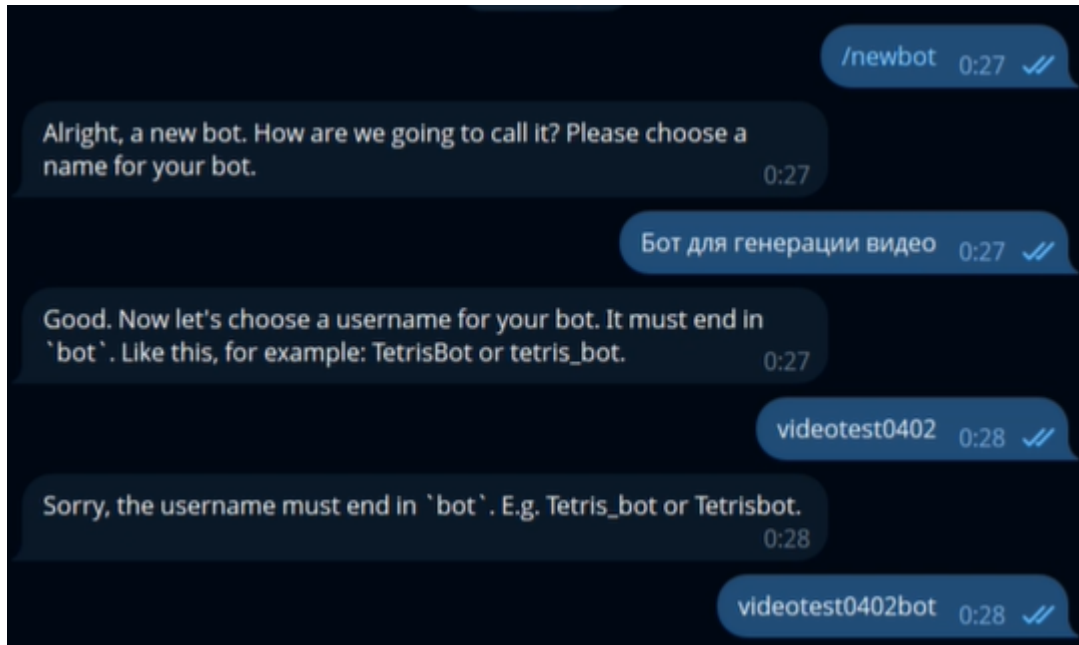
Перед тем как разбирать код, нужно создать Telegram-бота, потому что без него мы просто не сможем продолжать работу, и получить токен для соединения с ним.

1. Для этого заходим в Telegram, открываем бота BotFather, переходим по его юзернейму.

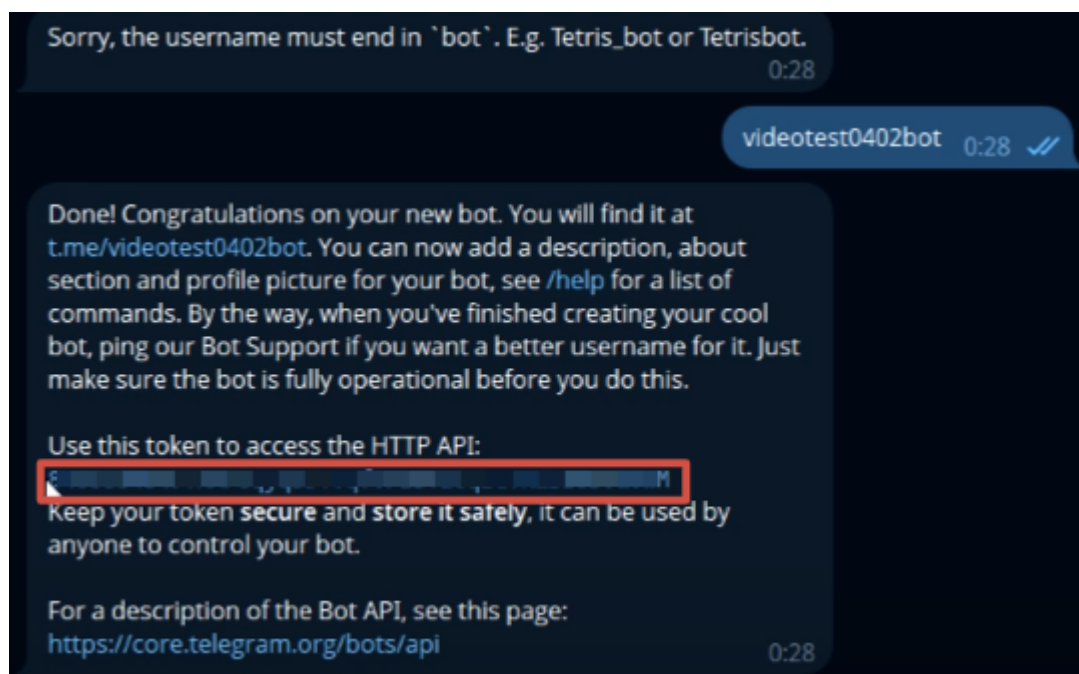


2. Запускаем диалог. Вводим команду создания нового бота `/newbot`.

3. Сначала нас просят ввести имя — это просто отображаемое название, например «Бот для генерации видео».
4. Дальше нужен юзернейм. Он должен быть уникальным и обязательно должен заканчиваться на `bot`. Если этого не сделать, Telegram просто не даст создать бота. Если забыть приписать `bot` в конце, в регистрации будет отказано.



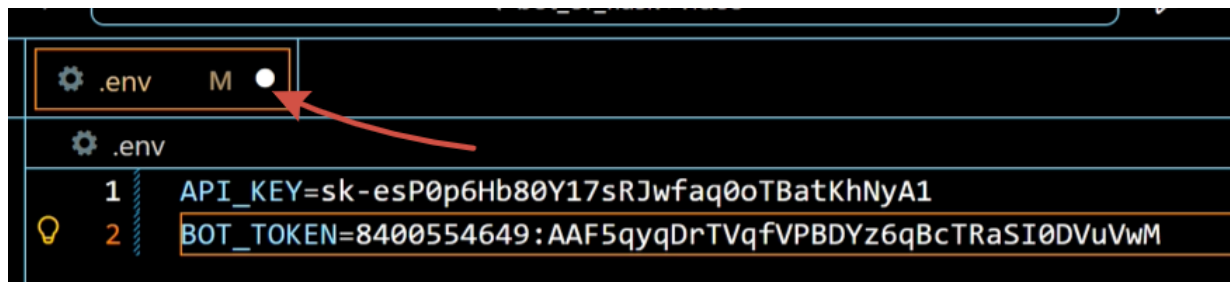
5. После этого мы получаем токен — длинную строку. Это и есть наш HTTP API токен. Копируем его.



6. Переходим в `.env` -файл внутри проекта и вставляем этот токен в переменную для бота после знака равенства:

```
BOT_TOKEN=
```

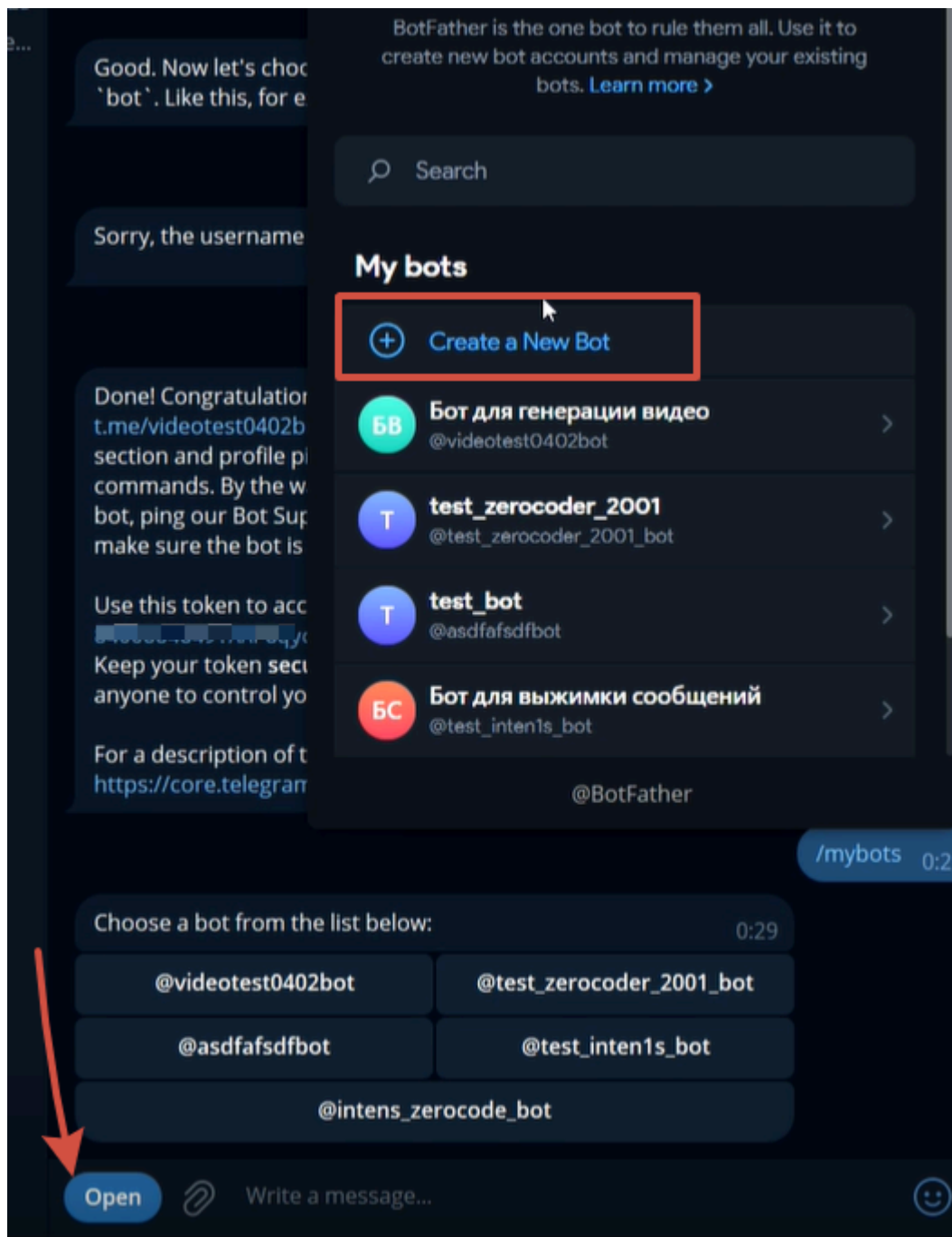
7. Видим значок (точку) рядом с названием файла: мы должны самостоятельно сохранить изменения. Это можно сделать сочетанием клавиш `Ctrl+S`.



8. Теперь можно проверить, что бот действительно создан — зайти в Telegram и найти его по имени.

💡 Если бот уже был создан раньше и мы забыли токен, можно в диалоге с BotFather ввести команду `/mybots`, выбрать нужного бота и снова посмотреть или скопировать API-токен.

9. Есть ещё один способ создать бота: нажать кнопку `Open` слева от диалога с BotFather — откроется мини-приложение. Нажимаем **Create a New Bot**.



10. Заполняем форму: в поле **Bot Name** вводим название, в поле **Username** — уникальный юзернейм с окончанием на bot и нажимаем Create Bot .

11. В следующем окне система выдаст токен (в скрытой форме); нажимаем Сору ; бот создан, токен получен.

Возвращаемся к проекту.

▼ Файл `request.py`

В файле `env` , кроме токена для бота, должен быть вставлен ключ для API (в нашем случае Proxy API. Вставляем его после знака равенства:

```
API_KEY=
```

Теперь перейдём к разбору файла `request.py` . Он почти не изменился, но здесь появилась важная конструкция:

```
if __name__ == "__main__":  
    generate_video("Крупный план чашки горячего кофе на деревянном с
```

Зачем нужна эта конструкция?

- Если мы запускаем файл напрямую командой `python request.py`, то значение `__name__` будет равно `"__main__"`, и код внутри этого блока выполнится.
- Но если мы импортируем этот файл из другого места (например, из `bot.py`), то этот блок уже не выполнится автоматически.

Это критически важно, потому что в `bot.py` мы импортируем функцию генерации:

```
from request import generate_video
```

Если бы не было условия `if __name__ == "__main__"`, то каждый раз при импорте файл `request.py` запускался бы целиком. А значит, каждый импорт приводил бы к новой генерации видео.

Это плохо по двум причинам: это долго, и каждый запрос стоит денег (и не копейки, а вполне реальные суммы). Поэтому эта конструкция нужна, **чтобы генерация запускалась только тогда, когда мы явно хотим её запустить**, а не при каждом импорте.

▼ Файл `bot.py`

Переходим к самому боту (файл `bot.py`).

1. В начале загружаем переменные окружения:

```
from dotenv import load_dotenv

load_dotenv()
```

2. Затем получаем токен бота:

```
BOT_TOKEN = os.getenv("BOT_TOKEN")

if not BOT_TOKEN:
    raise ValueError("BOT_TOKEN не найден в переменных окружения. Добавьте
```

Если токен не найден — выдаётся ошибка. Это защита от ситуации, когда мы забыли прописать его в `.env`.

3. Создаём объект бота:

```
bot = telebot.TeleBot(BOT_TOKEN)
```

Это наш основной объект, который принимает сообщения, обрабатывает их и отправляет ответы.

4. Далее создаём хранилище задач пользователей:

```
user_tasks = {}
```

Это обычный словарь, где ключ — это `user_id`, а значение — данные о текущей задаче пользователя: статус, прогресс, id видео и так далее.

Это нужно, чтобы если один пользователь попросил сгенерировать котёнка, а другой — щенка, мы не перепутали результаты и отправили каждому именно его видео.

5. Далее прописываем саму функцию генерации видео с отслеживанием прогресса для Telegram:

```
def generate_video_with_progress(prompt, user_id, message_id):  
    """Генерация видео с отслеживанием прогресса для Telegram"""
```

6. Здесь же создаём клиента OpenAI:

```
client = OpenAI(  
    api_key=f"{os.getenv('API_KEY')}",  
    base_url="<https://api.proxyapi.ru/openai/v1>",  
)
```

7. Начинаем процесс генерации:

```

try:
    video = client.videos.create(
        model="sora-2",
        prompt=f"{prompt}",
        seconds="4",
    )

```

Этот код мы уже разбирали в прошлом уроке. Добавился здесь фрагмент `user_tasks`:

```

user_tasks[user_id] = {
    "status": "started",
    "progress": 0,
    "message": "Генерация видео началась",
    "video_id": video.id
}

```

Это словарь формата «ключ — значение», внутри которого составляем пары с ID пользователя с данными о задаче, которую он запустил. Параметр `video.id` нужен, чтобы мы могли связаться ID пользователя и ID видео, чтобы прислать пользователю именно его видео.

8. Далее прописана функция обновления прогресс-бара:

```

def update_progress_message(user_id, message_id, status, progress, message):
    """Обновление сообщения с прогресс-баром"""
    try:
        bar_length = 20
        filled_length = int((progress / 100) * bar_length)
        bar = "█" * filled_length + "░" * (bar_length - filled_length)

```

Здесь мы:

- задаём длину прогресс-бара;
- считаем, сколько символов заполнено;
- формируем строку вида `█.....`;
- обновляем существующее сообщение, а не шлём новое.

9. Дальше прописана функция генерации видео с отслеживанием статуса:

```
status_emoji = {
    "started": "🚀",
    "queued": "⌚",
    "in_progress": "⚙️",
    "downloading": "⬇️",
    "completed": "✅",
    "failed": "❌",
    "error": "❌"
}

emoji = status_emoji.get(status, "⌚")
```

10. Таким образом, мы создаём клиента OpenAI и запускаем генерацию.
Сохраняем начальное состояние ("progress: 0"). Дальше запускаем цикл:

```
while video.status in ("in_progress", "queued"):
    video = client.videos.retrieve(video.id)
    progress = getattr(video, "progress", 0)

    status_text = "В очереди" if video.status == "queued" else "Обработка"
```

11. Пока статус `queued` или `in_progress` , мы:

- обновляем процент;
- обновляем сообщение пользователю;
- ждём 5 секунд и проверяем снова;
- сохраняем ID видео.

```
user_tasks[user_id] = {
    "status": video.status,
    "progress": progress,
    "message": status_text,
    "video_id": video.id
}

update_progress_message(user_id, message_id, video.status, pro
time.sleep(5)
```

12. Если статус `failed`, пишем пользователю, что произошла ошибка:

```
if video.status == "failed":
    message = getattr(
        getattr(video, "error", None), "message", "Генерация видео
    )
    user_tasks[user_id] = {
        "status": "failed",
        "progress": 100,
        "message": message,
        "video_id": video.id
    }
    update_progress_message(user_id, message_id, "failed", 100, me
```

13. Если всё успешно, меняем статус на `downloading`, скачиваем видео и отправляем его:

```
else:
    user_tasks[user_id] = {
        "status": "downloading",
        "progress": 95,
        "message": "Скачивание видео...",
        "video_id": video.id
    }
    update_progress_message(user_id, message_id, "downloading", 95
```

14. Отправляем готовое видео: открываем видеофайл и отправляем его методом `send_video`:

```
# Отправляем готовое видео
try:
    with open(video_path, 'rb') as video_file:
        bot.send_video(user_id, video_file, caption="✅ Видео")
```

15. Также добавлена обработка ошибок, которая поможет избежать проблем:

```
except Exception as e:
    user_tasks[user_id] = {
        "status": "error",
        "progress": 0,
        "message": f"Ошибка: {str(e)}",
        "video_id": None
    }
    update_progress_message(user_id, message_id, "error", 0, f"❌ Ошибка")
```

16. Также здесь мы выбираем подходящий эмодзи и подходящий текст:

```
emoji = status_emoji.get(status, "⌚")

progress_text = f"{emoji} {message_text}\n\n[{bar}] {progress:.1f}"
```

17. И обновляем сообщение пользователю:

```
bot.edit_message_text(
    progress_text,
    chat_id=user_id,
    message_id=message_id
)
```

Таким образом, мы не будем отправлять пользователю несколько сообщений подряд, а редактируем уже отправленное сообщение.

18. Далее прописаны стандартные обработчики — начала работы:

```
@bot.message_handler(commands=['start', 'help'])
def send_welcome(message):
    """Обработчик команд /start и /help"""
    welcome_text = """
    🖥 Добро пожаловать в бота для генерации видео через ИИ!
```

19. И обработка обычного текста:

```
@bot.message_handler(func=lambda message: True)
def handle_message(message):
    """Обработчик всех текстовых сообщений"""
    prompt = message.text.strip()
```

Любое текстовое сообщение — это запрос на генерацию.

20. В самом конце кода снова знакомая конструкция:

```
if __name__ == '__main__':
    print("Бот запущен...")
    bot.infinity_polling()
```

Это нужно, чтобы бот запускался, только когда мы запускаем файл напрямую командой `python bot.py`, а не при импорте в другие части проекта. Мы рекомендуем добавлять подобную конструкцию во все проекты.

▼ Запуск проекта (ТГ-бота)

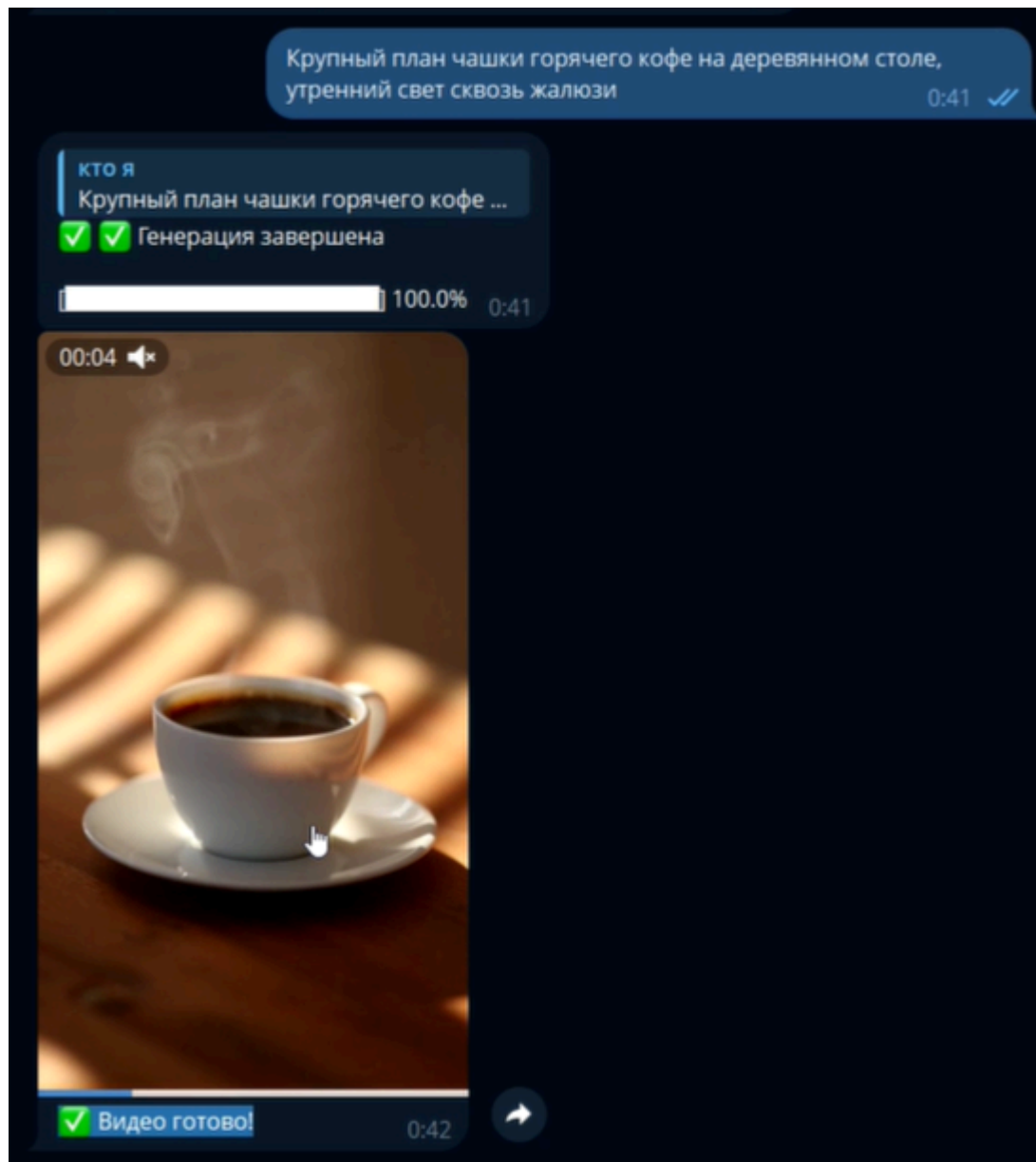
1. Активируем виртуальное окружение командой `.\venv\Scripts\activate`.
2. Устанавливаем зависимости командой `pip install -r requirements.txt`.

💡 У эксперта все зависимости уже установлены, но обращаем внимание, что в `requirements` появились новые библиотеки: `flask` и `TelegramBot API` — тебе нужно будет их установить командой, прописанной выше.

3. Запускаем бота командой `python .\bot.py`.

4. Переходим в Telegram, нажимаем Start , отправляем текст — и видим:

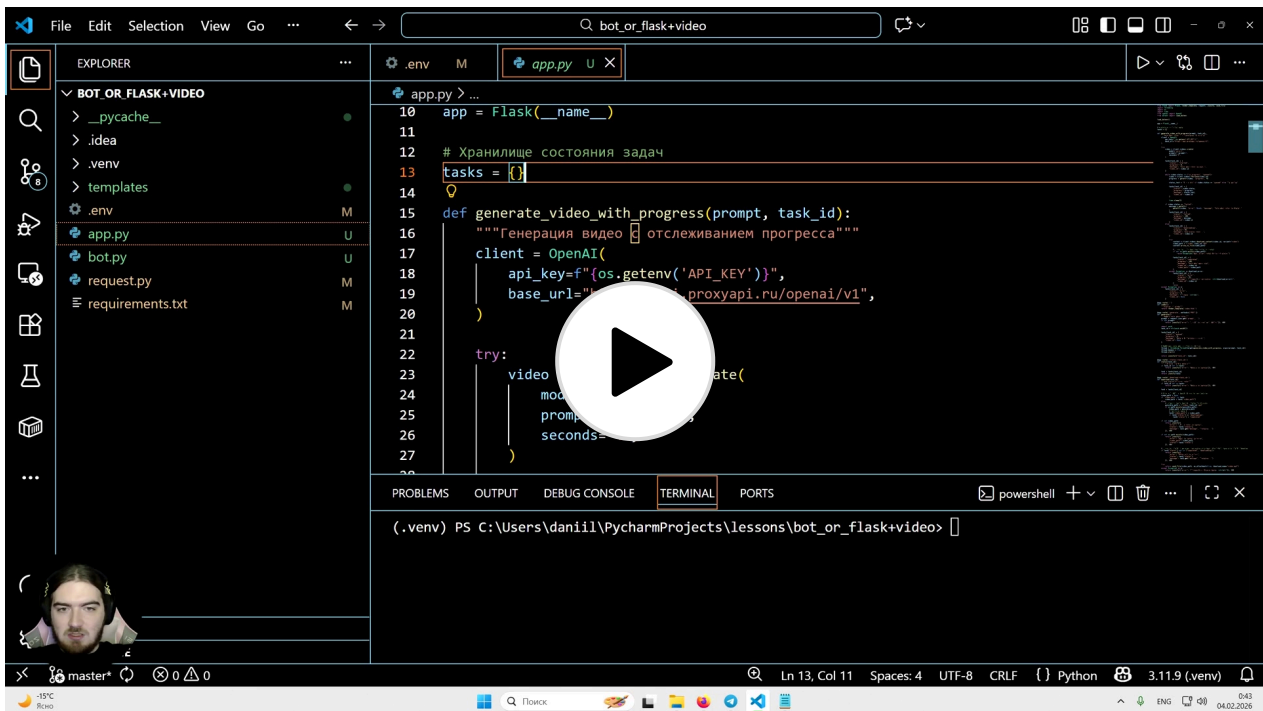
- сообщение о начале генерации;
- прогресс-бар;
- и наконец, готовое видео прямо внутри чата.



Это не файл, а именно видео, которое можно посмотреть, переслать или скачать. Бот работает корректно.

5. Возвращаемся в проект, останавливаем бота через сочетание клавиш `Ctrl+C` и очищаем консоль командой `clear` .

5. Реализация сайта для работы с генерацией видео



▼ Файл app.py

Переходим к разбору файла app.py.

1. Сначала подключаем переменные окружения, создаём объект Flask и заводим хранилище задач:

```
load_dotenv()

app = Flask(__name__)

# Хранилище состояния задач
tasks = {}
```

2. Дальше прописываем основную функцию генерации видео с прогрессом. Здесь создаётся клиент OpenAI:

```
def generate_video_with_progress(prompt, task_id):
    """Генерация видео с отслеживанием прогресса"""
    client = OpenAI(
        api_key=os.getenv('API_KEY'),
        base_url="https://api.proxyapi.ru/openai/v1",
    )
```

3. Отправляем запрос на генерацию видео:

```
video = client.videos.create(  
    model="sora-2",  
    prompt=f"{prompt}",  
    seconds="4",  
)
```

4. Сохраняем задачу: ей присваивается `task_id` и `video_id`.

```
tasks[task_id] = {  
    "status": "started",  
    "progress": 0,  
    "message": "Генерация видео началась",  
    "video_id": video.id  
}
```

Здесь нужно отметить два важных аспекта:

- В Telegram-боте у нас есть готовый уникальный идентификатор пользователя — это `user_id`. Telegram тем самым решает проблему идентификации: каждый пользователь уже имеет свой уникальный ID, и мы просто используем его как ключ для хранения задач.

В формате сайта такого механизма нет. У нас нет встроенного пользовательского ID, поэтому его надо создать самостоятельно. Для этого мы используем `task_id`, который передаём пользователю, и пользователь хранит его у себя. Когда он обращается к сайту (проверяет статус или пытается скачать видео), мы по этому `task_id` находим нужную задачу в нашем хранилище.

По сути, логика здесь ровно такая же, как и в Telegram-боте: есть идентификатор, по которому мы читаем состояние задачи.

Просто в боте этот идентификатор даёт Telegram, а на сайте мы генерируем его сами.

- В случае с Telegram-ботом после отправки файла пользователю в скрипте прописано его удаление. Мы его сгенерировали, прочитали, отправили пользователю — и дальше хранить его нет смысла, потому что он уже лежит у пользователя в чате.

В формате сайта так сделать нельзя. Здесь файл должен храниться и после генерации, потому что пользователь может не сразу нажать Скачать. Он может обновить страницу, вернуться позже, потерять вкладку и снова открыть ссылку. Поэтому в веб-версии видео хранится на сервере до тех пор, пока пользователь его не скачает. Здесь уже другая логика: не «одноразовая отправка», а постоянный доступ по `task_id`.

```
try:
    content = client.videos.download_content(video.id, varia
    video_path = f"video_{task_id}.mp4"
    content.write_to_file(video_path)
```

5. Далее проверяем, что файл действительно создан:

```
if not os.path.exists(video_path):
    raise Exception("Файл не был создан после сохранения")
```

6. Прописываем успешное выполнение:

```
tasks[task_id] = {
    "status": "completed",
    "progress": 100,
    "message": "Генерация завершена",
    "video_id": video.id,
    "video_path": video_path
}
```

7. Далее в коде прописаны роуты. Главная страница просто возвращает `index.html`:

```
@app.route('/')
def index():
    """Главная страница"""
    return render_template('index.html')
```

8. Роут для запуска генерации: получаем промпт и создаём уникальный `task_id` через UUID (функционал генерации уникального идентификатора).

```
@app.route('/generate', methods=['POST'])
def generate():
    """Запуск генерации видео"""
    prompt = request.json.get('prompt', '')

    import uuid
    task_id = str(uuid.uuid4())
```

✓ **UUID** это уникальный идентификатор задачи, который генерируется для пользователя на сайте, чтобы отличать его запрос от всех остальных.

Он состоит из букв, цифр и символов, сочетание которых с очень высокой вероятностью никогда не повторится. За счёт этого каждый пользователь получает свой собственный `task_id`, и мы можем по нему:

- хранить состояние задачи;
- проверять прогресс генерации;
- находить нужное видео;
- отдавать файл на скачивание.

9. Сохраняем задачу в очереди:

```
tasks[task_id] = {
    "status": "queued",
    "progress": 0,
    "message": "Задача поставлена в очередь",
    "video_id": None
}
```

10. Запускаем генерацию в отдельном потоке:

```
thread = threading.Thread(target=generate_video_with_progress, args=(p
thread.daemon = True
thread.start()
```

11. Возвращаем пользователю task_id:

```
return jsonify({"task_id": task_id})
```

12. Прописываем роут для проверки статуса задачи:

```
@app.route('/status/<task_id>')
def status(task_id):
    """Получение статуса задачи"""
    if task_id not in tasks:
        return jsonify({"error": "Задача не найдена"}), 404

    task = tasks[task_id]
    return jsonify(task)
```

13. И роут для скачивания видео:

```
@app.route('/download/<task_id>')
def download(task_id):
    """Скачивание готового видео"""
    if task_id not in tasks:
        return jsonify({"error": "Задача не найдена"}), 404

    task = tasks[task_id]
```

14. Проверяем, существует ли файл и разрешено ли скачивание:

```
if "video_path" in task:
    video_path = task["video_path"]
```

15. Обновляем задачу:

```
task["video_path"] = video_path
if task["status"] == "downloading":
    task["status"] = "completed"
```

16. Проверяем статус (разрешаем скачивание, если файл существует, даже если статус `downloading`):

```
if task["status"] not in ("completed", "downloading"):
    return jsonify({
        "error": "Видео ещё не готово",
        "status": task["status"],
        "message": task.get("message", "Ожидание...")
    }), 400
```

17. Отправляем пользователю файл с содержимым — видео mp4:

```
try:
    return send_file(video_path, as_attachment=True, download_name="vi
except Exception as e:
    return jsonify({"error": f"Ошибка при отправке файла: {str(e)}"}),
```

18. И прописываем стандартный запуск приложения:

```
if __name__ == '__main__':
    app.run(debug=True, port=5000)
```

▼ Запуск проекта (на сайте)

Переходим к развертыванию проекта на сервере.

1. Запускаем скрипт командой `python .\app.py`.
2. В консоли появляется ссылка на сайт. Переходим по ссылке.
3. Прописываем какой-нибудь промпт и нажимаем **Создать видео** :



Генератор видео через ИИ

Создайте видео из текстового описания

Описание видео:

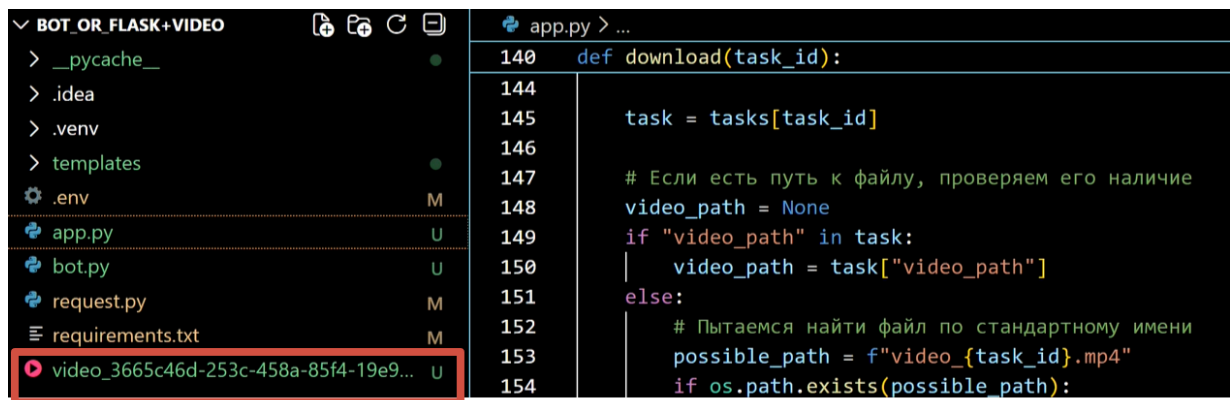
Котик на фоне зимнего леса

Создать видео

- Видим ошибку: для запуска запроса не хватает баланса. В этом случае нужно пополнить баланс, обновить страницу и попробовать сгенерировать промпт заново.
- На этот раз запуск прошёл успешно. Возвращаемся в консоль: видим здесь status 200 (успешный) и набор символов из букв, цифр и случайным образом распределённых тире: это и есть ID пользователя, о котором мы говорили выше.

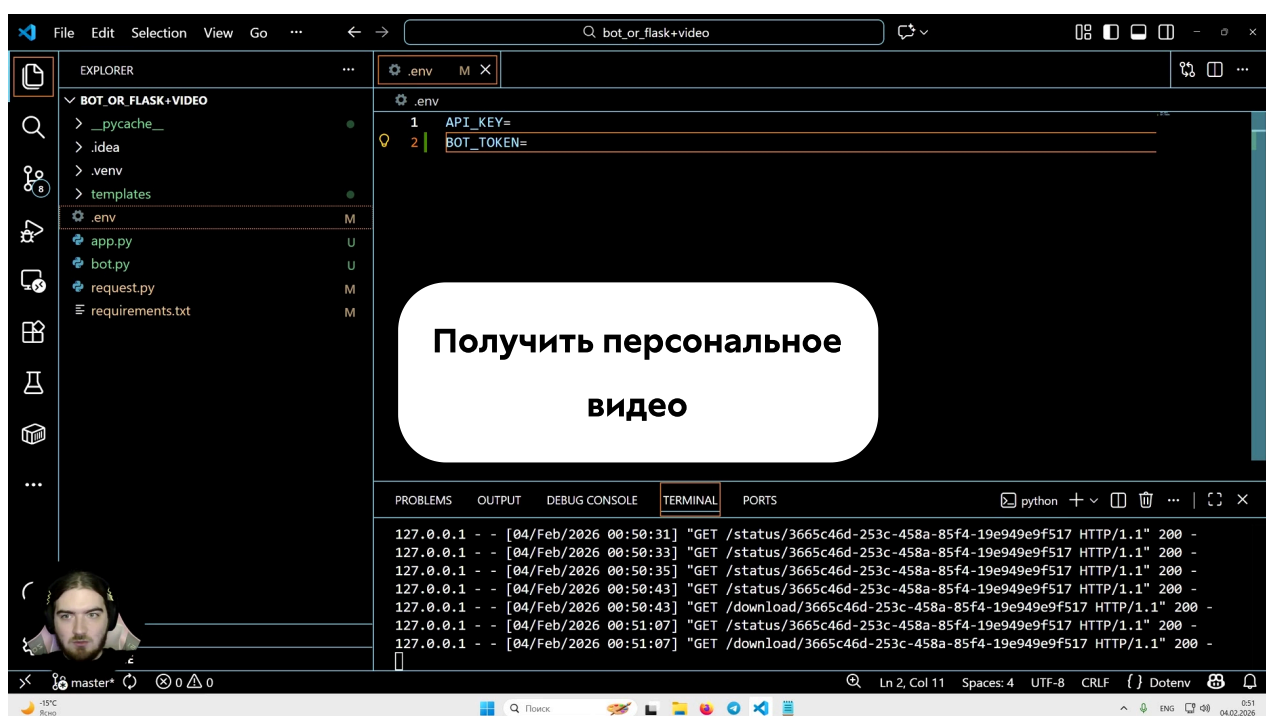
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  python + v [icon] [icon] ...
127.0.0.1 - - [04/Feb/2026 00:49:34] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [04/Feb/2026 00:49:37] "POST /generate HTTP/1.1" 200 -
127.0.0.1 - - [04/Feb/2026 00:49:39] "GET /status/3665c46d-253c-458a-85f4-19e949e9f517 HTTP/1.1" 200 -
127.0.0.1 - - [04/Feb/2026 00:49:42] "GET /status/3665c46d-253c-458a-85f4-19e949e9f517 HTTP/1.1" 200 -
127.0.0.1 - - [04/Feb/2026 00:49:44] "GET /status/3665c46d-253c-458a-85f4-19e949e9f517 HTTP/1.1" 200 -
127.0.0.1 - - [04/Feb/2026 00:49:46] "GET /status/3665c46d-253c-458a-85f4-19e949e9f517 HTTP/1.1" 200 -
127.0.0.1 - - [04/Feb/2026 00:49:48] "GET /status/3665c46d-253c-458a-85f4-19e949e9f517 HTTP/1.1" 200 -
127.0.0.1 - - [04/Feb/2026 00:49:50] "GET /status/3665c46d-253c-458a-85f4-19e949e9f517 HTTP/1.1" 200 -
```

- Проверяем задачу на сайте и видим, что прогресс идёт — прогресс-бар постепенно заполняется. Раз в несколько секунд мы обновляем его, отслеживаем, как идёт генерация, и наконец видим, что обработка достигает 99 процентов. В этот момент начинается скачивание файла.
- Наконец файл сгенерирован; на сайте нажимаем кнопку **Скачать**, и видео сохраняется на сервере и затем отправляется клиенту (оно также появляется в проекте на панели слева).



```
app.py > ...
140 def download(task_id):
141
142     task = tasks[task_id]
143
144     # Если есть путь к файлу, проверяем его наличие
145     video_path = None
146     if "video_path" in task:
147         video_path = task["video_path"]
148     else:
149         # Пытаемся найти файл по стандартному имени
150         possible_path = f"video_{task_id}.mp4"
151         if os.path.exists(possible_path):
```

8. Открываем видео; всё сгенерировано в соответствии с текстовым запросом. При желании можно отправить запрос для генерации следующего видео.



▼ Развёртывание проекта на сервере

1. Возвращаемся в Visual Studio. Останавливаем выполнение скрипта через сочетание клавиш `Ctrl+c`.
2. Удаляем из проекта ненужные видео и очищаем ключи в файле `.env`, чтобы не переносить их с сервера и не отправлять на GitHub.

⚠ Мы не рекомендуем вообще отправлять файл `.env` на GitHub. Но в этом проекте мы оставляем этот файл как пример, чтобы было понятно, какие ключи нужно заполнять и какие им давать названия.

3. Сохраняем все изменения и делаем коммит с названием, например, «V1.0.0 with UI», после чего отправляем изменения на GitHub (нажимаем **Push**).



UI (User Interface — пользовательский интерфейс) — это внешний вид, визуальная и интерактивная часть цифрового продукта (сайта, приложения, программы), с которой взаимодействует пользователь.

4. Переходим на GitHub. Переходим в репозитории, видим, что проект загружен.



Ссылка на проект в GitHub есть в дополнительных материалах к этому уроку.

5. Возвращаемся в Visual Studio. Деактивируем виртуальное окружение (команда `deactivate`).
6. Подключаемся по SSH стандартным методом (`ssh root`).
7. При первом подключении к виртуальному серверу в консоли всегда возникнет вопрос, уверены ли мы, что хотим подключиться. Подтверждаем подключение (`yes`).
8. Система требует ввести пароль из Beget. Копируем пароль (он не будет отображаться в консоли, это нормально).
9. Очищаем терминал командой `clear` и выполняем команду `git clone` со ссылкой на проект GitHub.
10. Ждём, когда проект скачается. После скачивания проекта прописываем команду `ls` для просмотра содержимого и переходим в папку проекта (команда `cd video_generate/`).
11. Создаём виртуальное окружение: `python3 -m venv venv` .



Python3 — это стандартный инструмент Ubuntu-сервера, поэтому его можно использовать всегда.

12. Видим ошибку: на сервере не установлен инструмент для создания окружения. Устанавливаем его через команду `apt install python3.12-venv -y` (где `y` подтверждает, что мы действительно хотим установить инструмент).
13. Очищаем терминал. Запускаем виртуальное окружение (`python3 -m venv venv`).

14. Взглянем на все файлы проекта, в том числе скрытые: для этого прописываем команду `ls -a`. Видим эти файлы в консоли:

```
root@faxahkdouo:~/video_generate# python3 -m venv venv
root@faxahkdouo:~/video_generate# ls -a
.  ..  app.py  bot.py  .env  .git  .idea  __pycache__  request.py  requirements.txt  templates  venv
root@faxahkdouo:~/video_generate# ls
app.py  bot.py  __pycache__  request.py  requirements.txt  templates  venv
root@faxahkdouo:~/video_generate# ls -a
.  ..  app.py  bot.py  .env  .git  .idea  __pycache__  request.py  requirements.txt  templates  venv
root@faxahkdouo:~/video_generate#
```



Скрывать эти файлы нужно, чтобы не засорять вывод: с этими файлами мы практически не работаем.

15. Запускаем виртуальное окружение командой:

- для Mac-системы — `/source venv/bin/activate` ;
- для системы Windows — `venv\Scripts\Activate` .

16. Устанавливаем все необходимые библиотеки командой `pip install -r requirements.txt` .

17. Снова очищаем терминал.

18. Если запустить код напрямую (`python bot.py`), он будет работать только в этом терминале. Поэтому, чтобы обеспечить работу 24/7, используем команду `screen` : создаём виртуальный терминал командой `screen -S bot` .

19. Внутри `screen` запускаем виртуальное окружение.

20. Теперь необходимо отредактировать `env` -файл: ведь сейчас в нём нет ключей. Для этого прописываем команду `nano .env` .

21. Заполняем необходимыми данными `env` (API-ключ и токен бота). Для сохранения изменений нажимаем `Ctrl-X` → `Y` → `Enter` .

22. Запускаем бота (`python bot.py`).

23. Проверяем, запустился ли бот: переходим в Telegram, запускаем диалог командой `/start` . Видим, что всё работает.

24. Отправляем в диалог с ботом какой-нибудь текстовый запрос.

25. Сворачиваем терминал сочетанием клавиш `Ctrl-A-D` (он продолжит работу в фоновом режиме).

26. Создаём второй `screen` для сайта командой `screen -S site` .

27. Запускаем виртуальное окружение.

28. Запускаем файл `app` (команда `python app.py`).

29. Для работы сайта на сервере нужно немного отредактировать запуск: открываем файл (командой `nano app`) и добавляем в него параметр

host='0.0.0.0' и устанавливаем debug=False .

```
GNU nano 7.2
if __name__ == '__main__':
    app.run(host='0.0.0.0', debug=False, port=5000)
```

⚠ Без этого сайт будет доступен только по локальному адресу сервера:

- параметр host='0.0.0.0' позволяет принимать запросы со всех IP-адресов;
- параметр debug=False отключает показ ошибок обычному пользователю.

30. Внеся изменения в файл, нажимаем сочетание клавиш Ctrl+X → Y → Enter .

31. Очищаем терминал и запускаем файл (python app.py).

Теперь проект работает: в консоли появляется надпись: «Running on all addresses». Бот и сайт запущены и доступны.

▼ Тестирование работы сайта

1. Переходим по ссылке (наводим курсор на ссылку в консоли, зажимаем клавишу Ctrl и делаем клик левой кнопкой мыши).

Сайт открылся, видим форму для ввода запроса. В адресном окне видим IP-адрес удалённого сервера: он совпадает с IP-адресом, указанным в нашем профиле виртуального сервера Beget. Значит, мы действительно подключились к этому сайту и запустили проект.

2. Если мы выйдем с сервера или закроем терминал, уничтожим подключения, процессы в screen продолжают работать: и сайт, и бот отвечают на запросы.

3. Повторное подключение через SSH позволяет управлять этими терминалами: прописываем команду ssh root@ и дальше указываем IP-адрес виртуального сервера.

4. Вводим пароль. При повторном подключении не требуется подтверждения, действительно ли мы хотим подключиться.

5. Вводим команду screen -ls , которая показывает активные процессы: в нашем случае это два screen-терминала: site и bot.

```
root@faxahkdouo:~# screen -ls
There are screens on:
1544.site      (02/03/2026 09:59:40 PM)      (Detached)
1524.bot       (02/03/2026 09:58:17 PM)      (Detached)
2 Sockets in /run/screen/S-root.
root@faxahkdouo:~#
```

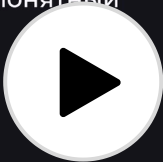
6. Чтобы подключиться к этим терминалам, прописываем команду `screen -r <название терминала>` (site или bot).
7. Чтобы прекратить выполнение процесса, нажимаем `Ctrl+C` и прописываем команду `exit`.

Таким образом, этих шагов достаточно, чтобы запустить проект на сервере, обеспечить его работу 24/7 и демонстрировать его заказчику или коллегам.

6. Результаты урока

Итоги урока

1. Повторили как мы можем делать запросы к различным нейросетям
2. Обернули наше приложение в понятный пользователю интерфейс
3. Завершили наш второй кейс







▼ Итоги урока

Сегодня мы:

- Повторили, как мы можем делать запросы к различным нейросетям;
- Повторили принцип работы открытых площадок для взаимодействия с множеством разных API и сервисов;

- Обернули наше приложение в понятный пользователю интерфейс;

💡 Этим пользователем можем быть и мы сами: например, можно создать собственный кейс, запустить ассистента на сервере, обернуть его в формат Telegram-бота или любой другой интерфейс, который нам удобен, и дальше работать с ним без постоянного доступа к компьютеру.

Через такие сценарии можно реализовать огромное количество разных идей. По сути, практически все сервисы, которыми мы пользуемся, устроены именно так.

- Завершили наш второй кейс, реализовав два варианта взаимодействия — через сайт и через Telegram-бот.

Вся система работает стабильно и запущена на сервере.

Мы также посмотрели команды, которые можно использовать для развёртывания проектов на сервере, чтобы распространять их, самостоятельно с ними работать, показывать коллегам или друзьям либо просто развернуть что-то для себя (например, собственного Telegram-бота, который будет выполнять роль ассистента или системы напоминаний).

В дальнейшем ты сможешь продолжать реализовывать похожие кейсы, занимаясь созданием ИИ-ассистентов. Ведь создать самого ассистента — это только половина задачи; вокруг него обязательно нужно построить удобную обёртку, через которую с ним можно будет общаться, и чтобы для пользователя всё это было понятно, просто и комфортно.

Дополнительные материалы

▼ Код проекта эксперта

Код проекта эксперта на GitHub

(https://github.com/MrGAN12009/video_generate)

► Ссылки на программы и сайты

Домашнее задание

В этом задании твоя задача — превратить AI-скрипт генерации видео в полноценный продукт с интерфейсом и развернуть его на сервере.

Часть 1. Telegram-бот

1. Создай собственного бота в Telegram через BotFather.
 2. Подключи токен бота через `.env`.
 3. Реализуй:
 - приём текстового запроса (промпта);
 - запуск генерации видео через backend-скрипт;
 - прогресс-бар со статусами (`queued` / `in_progress` / `downloading` / `completed` / `failed`);
 - отправку готового видео в чат;
 - обработку ошибок.
 4. Проверь, что бот:
 - корректно запускается,
 - не генерирует видео при импорте (используется `if __name__ == "__main__":`),
 - обновляет одно сообщение вместо отправки множества новых.
-

Часть 2. Flask-сайт

1. Создай веб-интерфейс на Flask:
 - поле ввода промпта,
 - кнопка запуска генерации,
 - отображение прогресса,
 - кнопка скачивания готового видео.
2. Реализуй:
 - генерацию `task_id` через UUID;
 - хранение статуса задачи;
 - роуты:

- /generate
- /status/<task_id>
- /download/<task_id>

3. Проверь:

- корректную работу потоков (threading),
- что видео сохраняется на сервере,
- что скачивание возможно даже после обновления страницы.

Часть 3. Развёртывание на сервере

1. Залей проект на GitHub.

2. Подключись к серверу по SSH.

3. Выполни:

- `git clone`
- создание виртуального окружения
- установку зависимостей

4. Запусти:

- бота в `screen`
- сайт в отдельном `screen`

5. Настрой запуск Flask с параметрами:

- `host='0.0.0.0'`
- `debug=False`

6. Проверь, что:

- бот отвечает,
- сайт открывается по IP,
- процессы продолжают работать после закрытия SSH-сессии.

Что нужно предоставить в качестве отчёта

Загрузи в систему следующие материалы:

1 Скриншоты

- созданного бота в Telegram;

- переписки с ботом (виден прогресс-бар и отправленное видео);
- работающего сайта (ввод промпта + отображение прогресса);
- страницы скачивания видео;
- активных screen-процессов на сервере (`screen -ls`);
- репозитория на GitHub.

2 Ссылка

- ссылка на GitHub-репозиторий.

3 Краткое текстовое описание (5–10 предложений)

Опиши:

- как устроена архитектура проекта (backend / frontend),
- чем отличается логика Telegram-бота от веб-версии,
- какие сложности возникли при развёртывании.

На следующем занятии

На следующем занятии мы создадим генератор постов для блога в Telegram с помощью n8n: разберём настройку workflow, интеграцию LLM для генерации текстов по промптам, автоматизацию публикации, обработку тем, стилей и расписания, и поймём, как это усиливает промпт-инжиниринг для контент-маркетинга.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PErr10.4 Генератор видео. Часть 2 (new)

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно

☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

В этом задании твоя задача — превратить AI-скрипт генерации видео в полноценный продукт с интерфейсом и развернуть его на сервере.

Часть 1. Telegram-бот

1. Создай собственного бота в Telegram через BotFather.
2. Подключи токен бота через `.env`.
3. Реализуй:
 - приём текстового запроса (промпта);
 - запуск генерации видео через backend-скрипт;
 - прогресс-бар со статусами (`queued` / `in_progress` / `downloading` / `completed` / `failed`);
 - отправку готового видео в чат;
 - обработку ошибок.
4. Проверь, что бот:
 - корректно запускается,
 - не генерирует видео при импорте (используется `if __name__ == "__main__":`),
 - обновляет одно сообщение вместо отправки множества новых.

Часть 2. Flask-сайт

1. Создай веб-интерфейс на Flask:

- поле ввода промпта,
- кнопка запуска генерации,
- отображение прогресса,
- кнопка скачивания готового видео.

2. Реализуй:

- генерацию `task_id` через UUID;
- хранение статуса задачи;
- роуты:
 - `/generate`
 - `/status/<task_id>`
 - `/download/<task_id>`

3. Проверь:

- корректную работу потоков (threading),
 - что видео сохраняется на сервере,
 - что скачивание возможно даже после обновления страницы.
-

Часть 3. Развёртывание на сервере

1. Залей проект на GitHub.

2. Подключись к серверу по SSH.

3. Выполни:

- `git clone`
- создание виртуального окружения
- установку зависимостей

4. Запусти:

- бота в `screen`
- сайт в отдельном `screen`

5. Настрой запуск Flask с параметрами:

- `host='0.0.0.0'`

- `debug=False`

6. Проверь, что:

- бот отвечает,
- сайт открывается по IP,
- процессы продолжают работать после закрытия SSH-сессии.



Что нужно предоставить в качестве отчёта

Загрузи в систему следующие материалы:

1 СКРИНШОТЫ

- созданного бота в Telegram;
- переписки с ботом (виден прогресс-бар и отправленное видео);
- работающего сайта (ввод промпта + отображение прогресса);
- страницы скачивания видео;
- активных screen-процессов на сервере (`screen -ls`);
- репозитория на GitHub.

2 ССЫЛКА

- ссылка на GitHub-репозиторий.

3 КРАТКОЕ ТЕКСТОВОЕ ОПИСАНИЕ (5–10 ПРЕДЛОЖЕНИЙ)

Опиши:

- как устроена архитектура проекта (backend / frontend),
- чем отличается логика Telegram-бота от веб-версии,
- какие сложности возникли при развёртывании.



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=341378470>)

Следующий урок